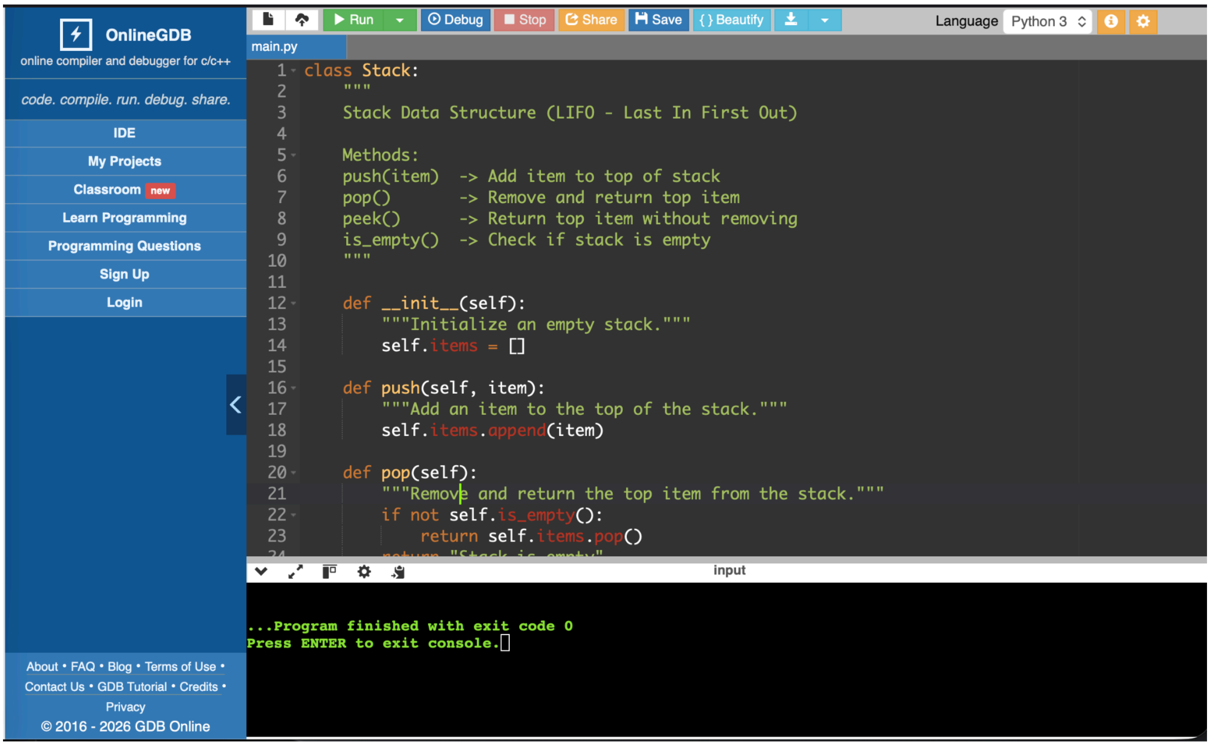
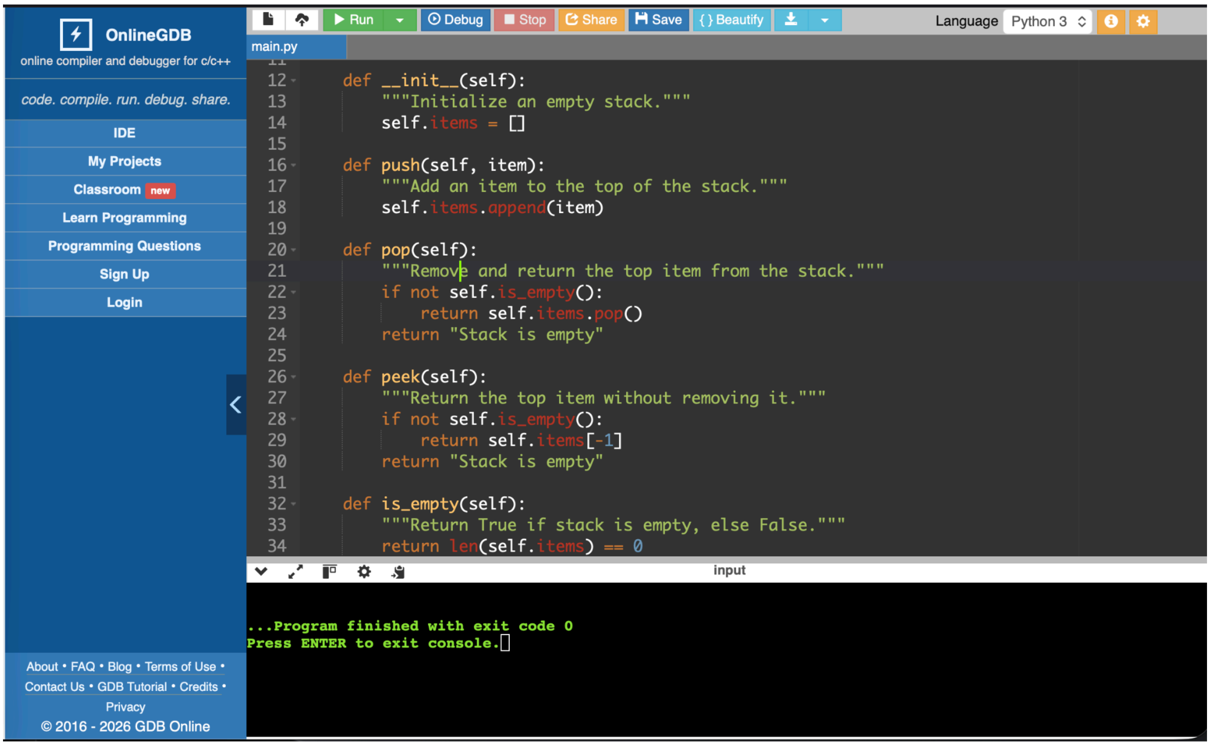
2303A52416
batch=35
Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
Sample Input Code:

```
class Stack:
pass
```

Expected Output:
• A functional stack implementation with all required methods and docstrings.
Code

```
class Stack:
    """
    Stack Data Structure (LIFO - Last In First Out)

    Methods:
    push(item)  -> Add item to top of stack
    pop()       -> Remove and return top item
    peek()      -> Return top item without removing
    is_empty()  -> Check if stack is empty
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item from the stack."""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return the top item without removing it."""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        """Return True if stack is empty, else False."""
        return len(self.items) == 0
```





Explanation
uses Python list internally.
Follows LIFO principle.
append() → push
pop() → remove top element

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.
Sample Input Code:

```
class Queue:
pass
```
Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code

```
class Queue:
    """
    Queue Data Structure (FIFO - First In First Out)

    Methods:
        enqueue(item)  -> Add item to the rear of the queue
        dequeue()      -> Remove and return the front item
        peek()         -> Return the front item without removing it
        size()         -> Return number of elements in the queue
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """
        Add an item to the rear of the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """
        Remove and return the front item of the queue.
        Raises IndexError if queue is empty.
        """
        if self.size() == 0:
            raise IndexError("Dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        """
        Return the front item without removing it.
        Raises IndexError if queue is empty.
        """
        if self.size() == 0:
            raise IndexError("Peek from empty queue")
        return self.items[0]

    def size(self):
        """
        Return the number of elements in the queue.
```

```
    """
    return len(self.items)

def is_empty(self):
    """
    Return True if queue is empty, otherwise False.
    """
    return self.size() == 0
```

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE · My Projects · Classroom new · Learn Programming · Programming Questions · Sign Up · Login

Run · Debug · Stop · Share · Save · { } Beautify — Language: Python 3

main.py

```python
class Queue:
    """
    Queue Data Structure (FIFO - First In First Out)

    Methods:
        enqueue(item)  -> Add item to the rear of the queue
        dequeue()      -> Remove and return the front item
        peek()         -> Return the front item without removing it
        size()         -> Return number of elements in the queue
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """
        Add an item to the rear of the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """
```

input

```
...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE · My Projects · Classroom new · Learn Programming · Programming Questions · Sign Up · Login

Run · Debug · Stop · Share · Save · { } Beautify — Language: Python 3

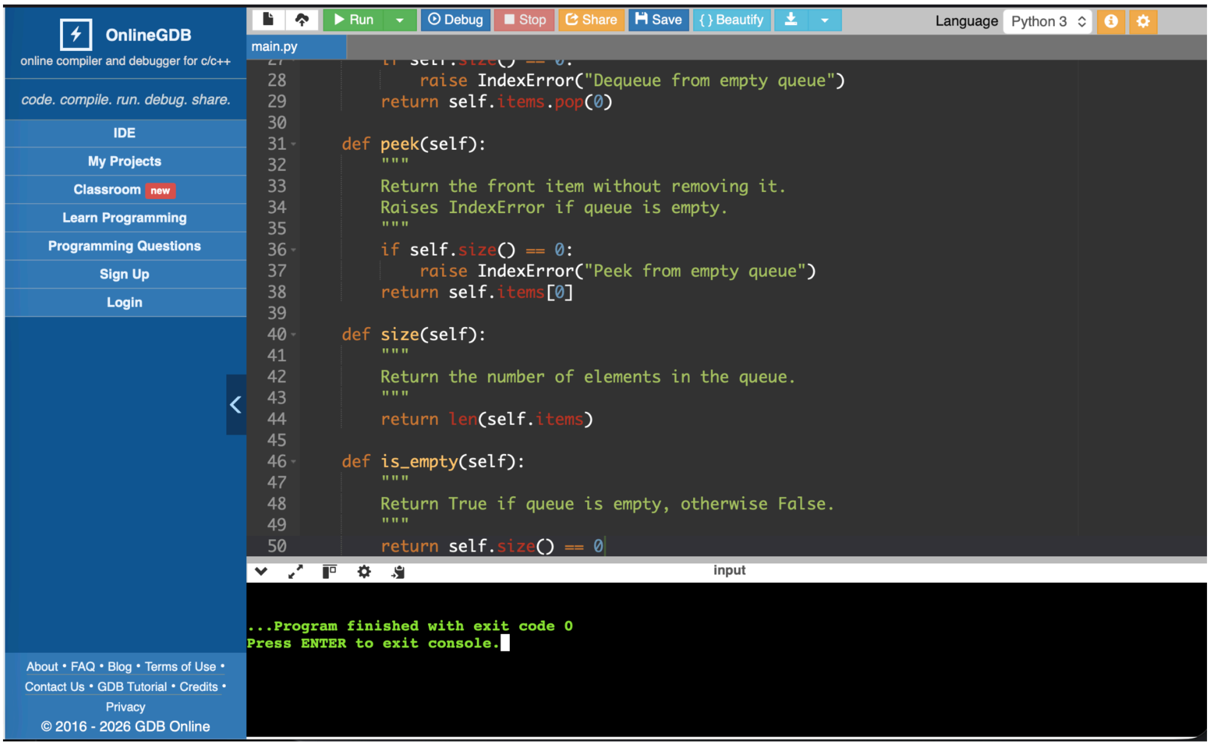
main.py

```python
        Remove and return the front item of the queue.
        Raises IndexError if queue is empty.
        """
        if self.size() == 0:
            raise IndexError("Dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        """
        Return the front item without removing it.
        Raises IndexError if queue is empty.
        """
        if self.size() == 0:
            raise IndexError("Peek from empty queue")
        return self.items[0]

    def size(self):
        """
        Return the number of elements in the queue.
        """
        return len(self.items)

    def is_empty(self):
```

input

```
...Program finished with exit code 0
Press ENTER to exit console.
```

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online



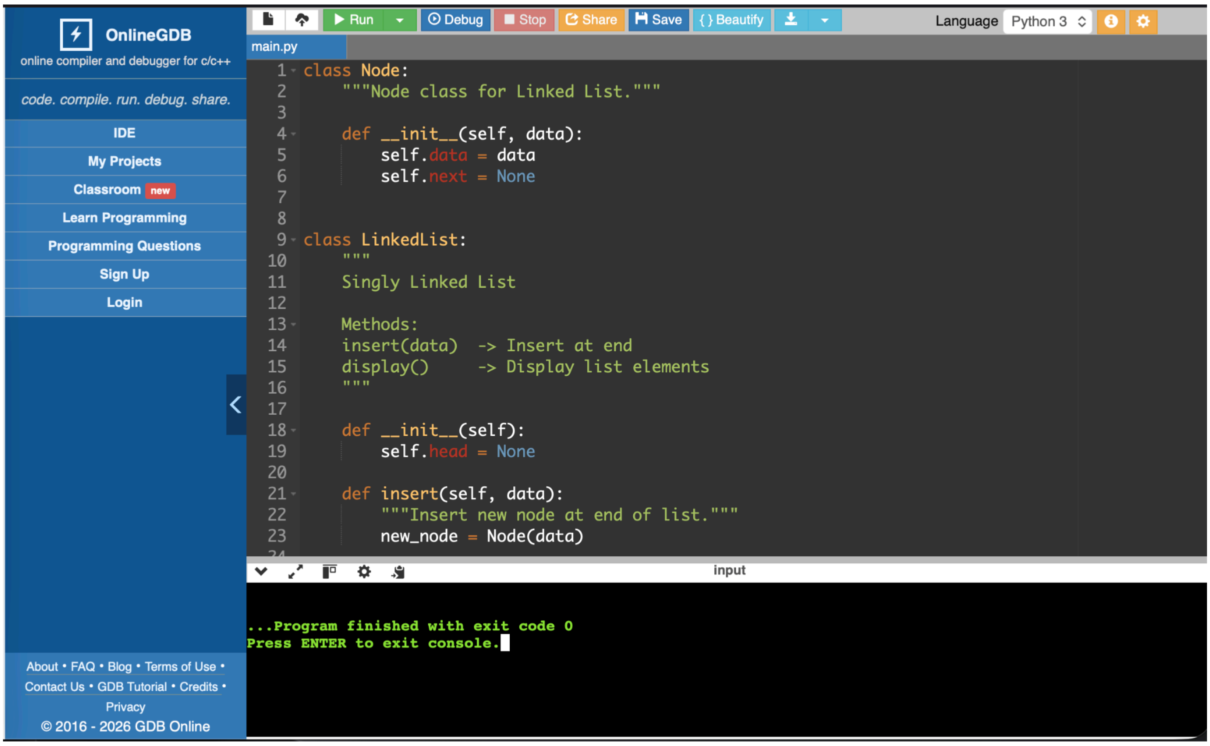
Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display methods.
Sample Input Code:
class Node:
pass
class LinkedList:
pass
Expected Output:
• A working linked list implementation with clear method documentation.
Code

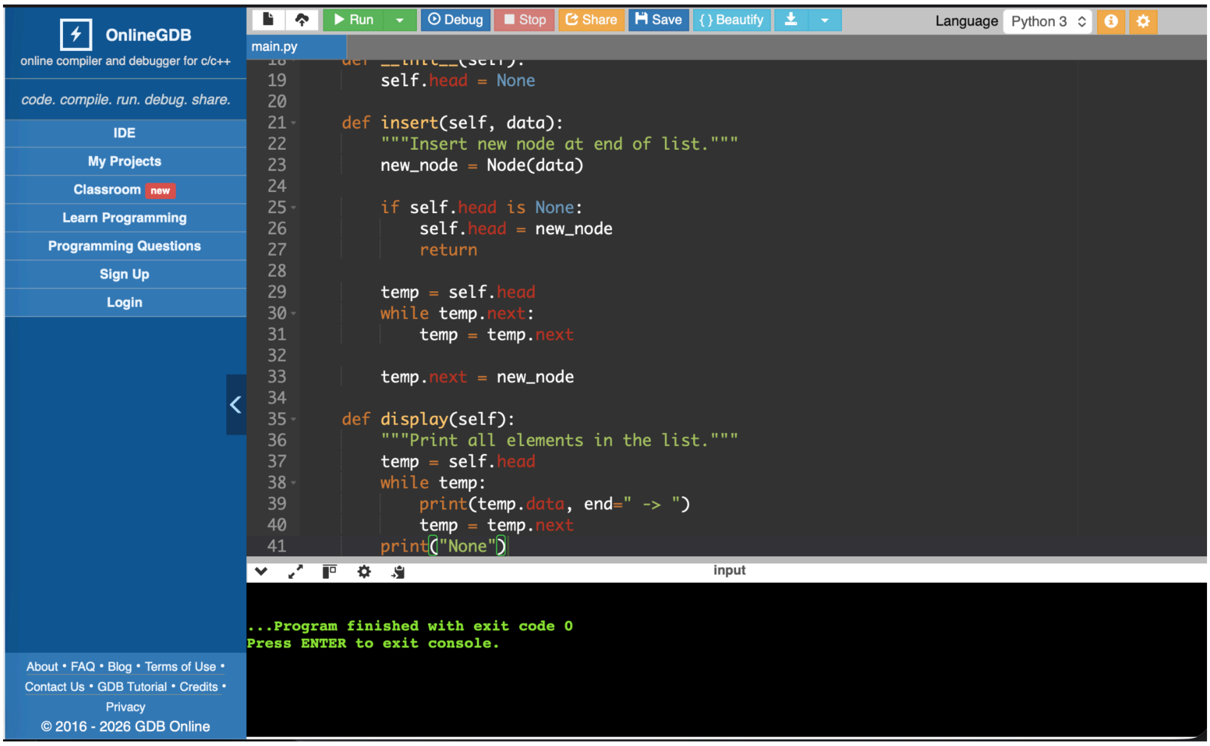
```
class Node:
    """Node class for Linked List."""

    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    """
    Singly Linked List

    Methods:
    insert(data)  -> Insert at end
    display()     -> Display list elements
```

```python
    """

    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert new node at end of list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    def display(self):
        """Print all elements in the list."""
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

Explanation

Follows FIFO principle.
append() → add at rear
pop(0) → remove from front





Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

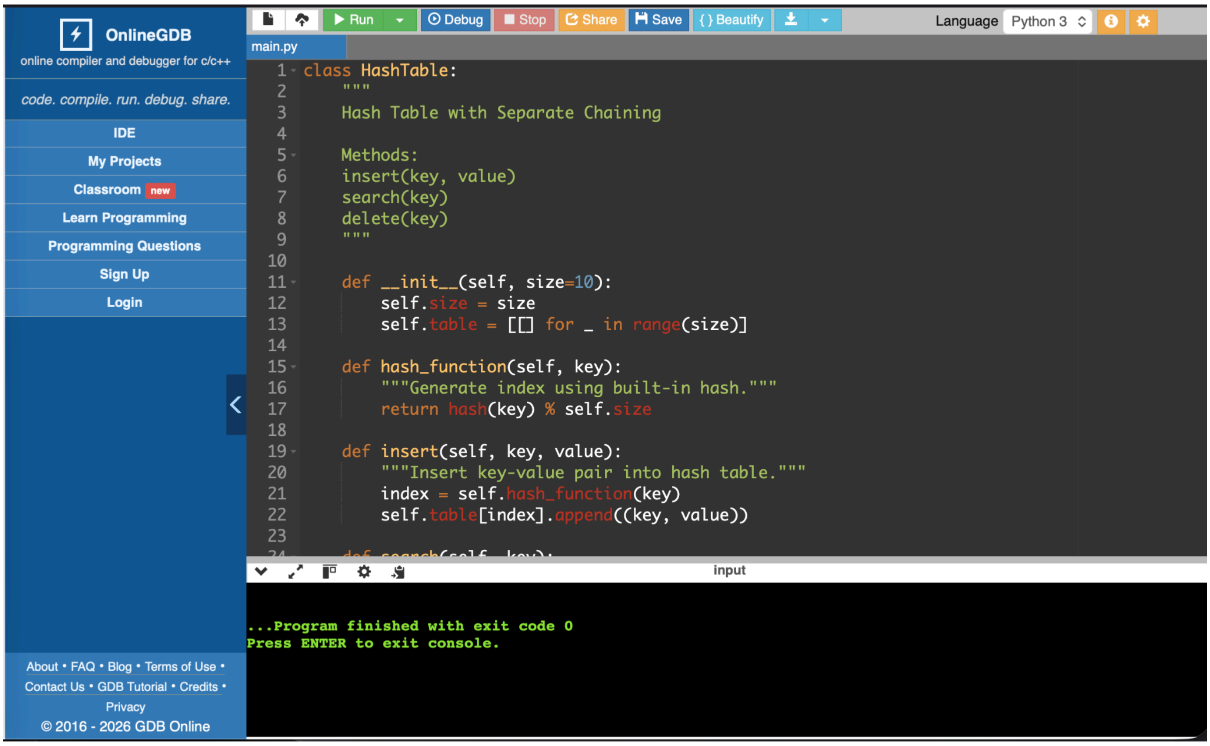
pass

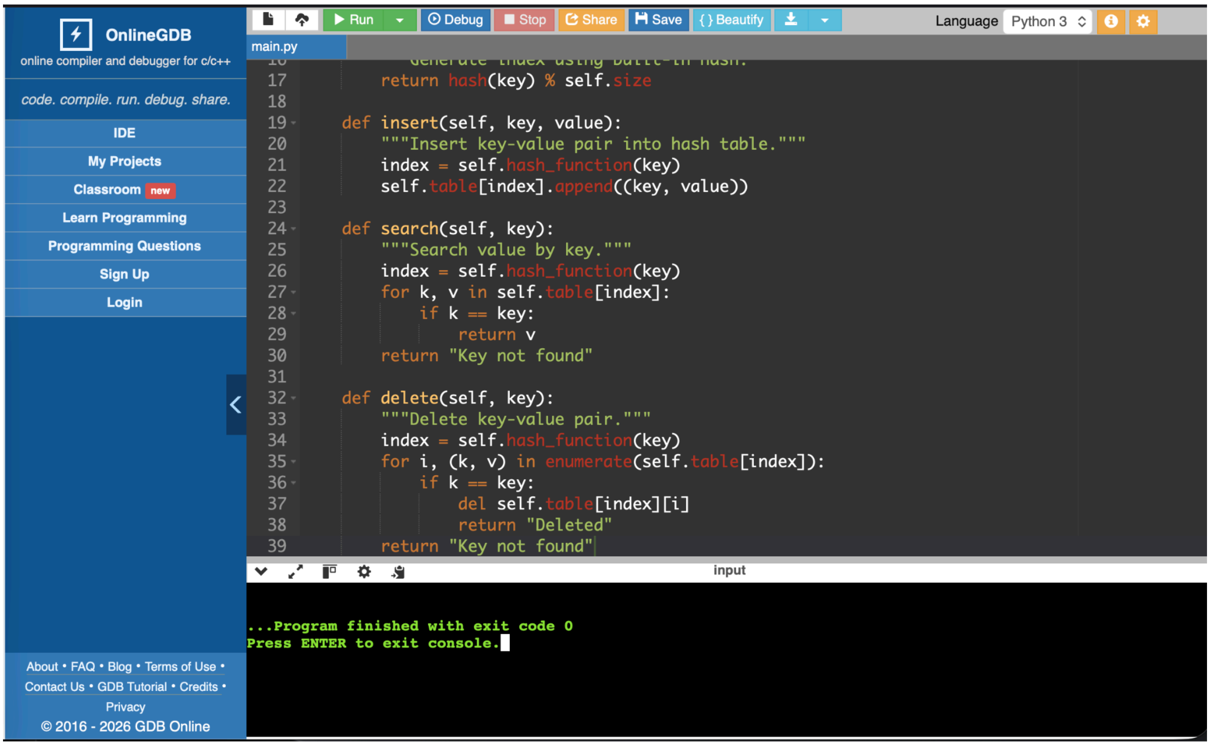
Expected Output:

• Collision handling using chaining, with well-commented methods.

Code

```python
class HashTable:
    """
    Hash Table with Separate Chaining

    Methods:
    insert(key, value)
    search(key)
    delete(key)
    """

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        """Generate index using built-in hash."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert key-value pair into hash table."""
        index = self.hash_function(key)
        self.table[index].append((key, value))

    def search(self, key):
        """Search value by key."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return "Key not found"

    def delete(self, key):
        """Delete key-value pair."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return "Deleted"
        return "Key not found"
```

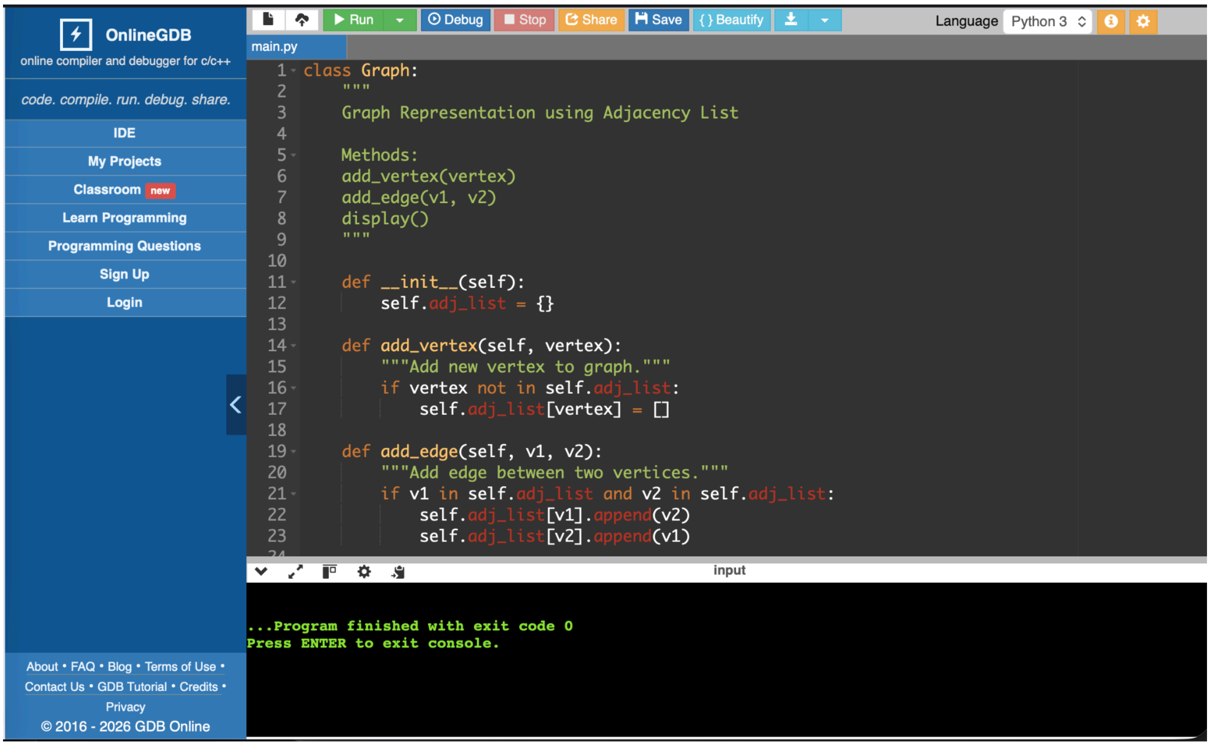


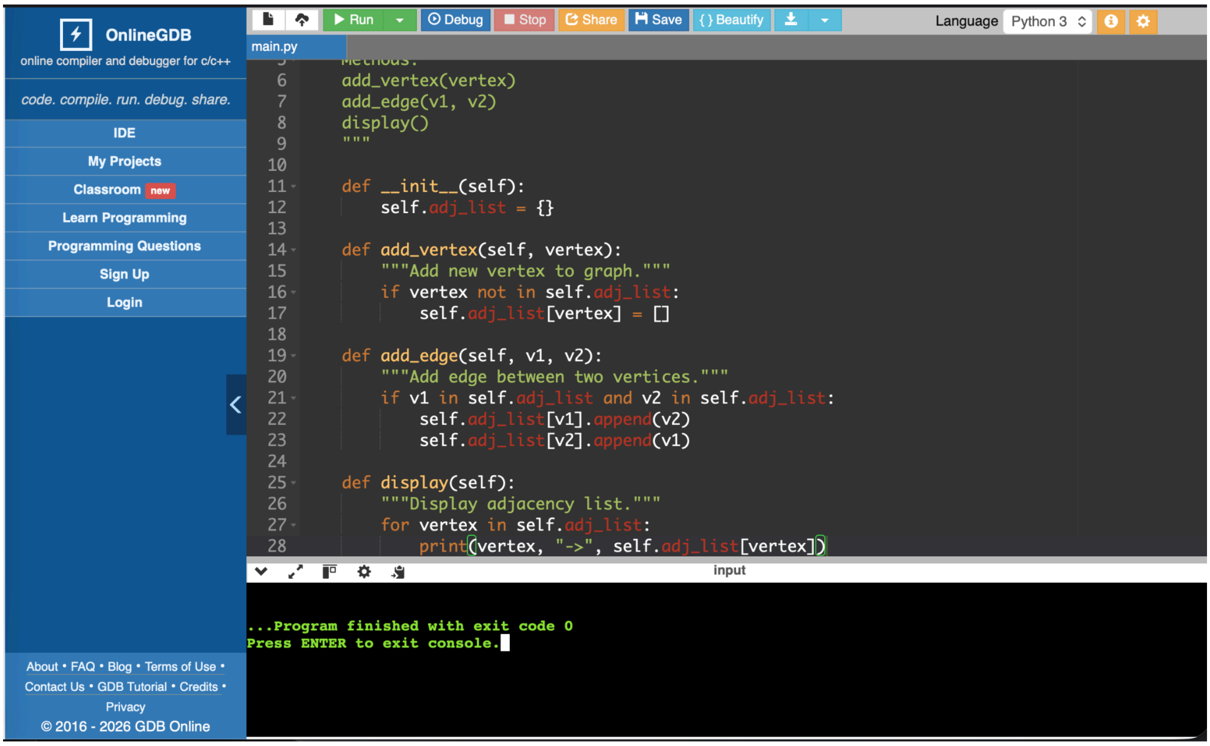


Explanation
Uses separate chaining for collision handling.
Hash function maps key to bucket index.
Each bucket is a list of key-value pairs.

Task Description #5 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph:

pass
Expected Output:
• Graph with methods to add vertices, add edges, and display connections.
Code
```
class Graph:
    """
    Graph Representation using Adjacency List

    Methods:
    add_vertex(vertex)
    add_edge(v1, v2)
    display()
    """

    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        """Add new vertex to graph."""
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, v1, v2):
        """Add edge between two vertices."""
        if v1 in self.adj_list and v2 in self.adj_list:
            self.adj_list[v1].append(v2)
            self.adj_list[v2].append(v1)

    def display(self):
        """Display adjacency list."""
        for vertex in self.adj_list:
            print(vertex, "->", self.adj_list[vertex])
```





Explanation
Dictionary stores adjacency list.
Efficient for sparse graphs.
Suitable for networks.

Structure Selection
A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

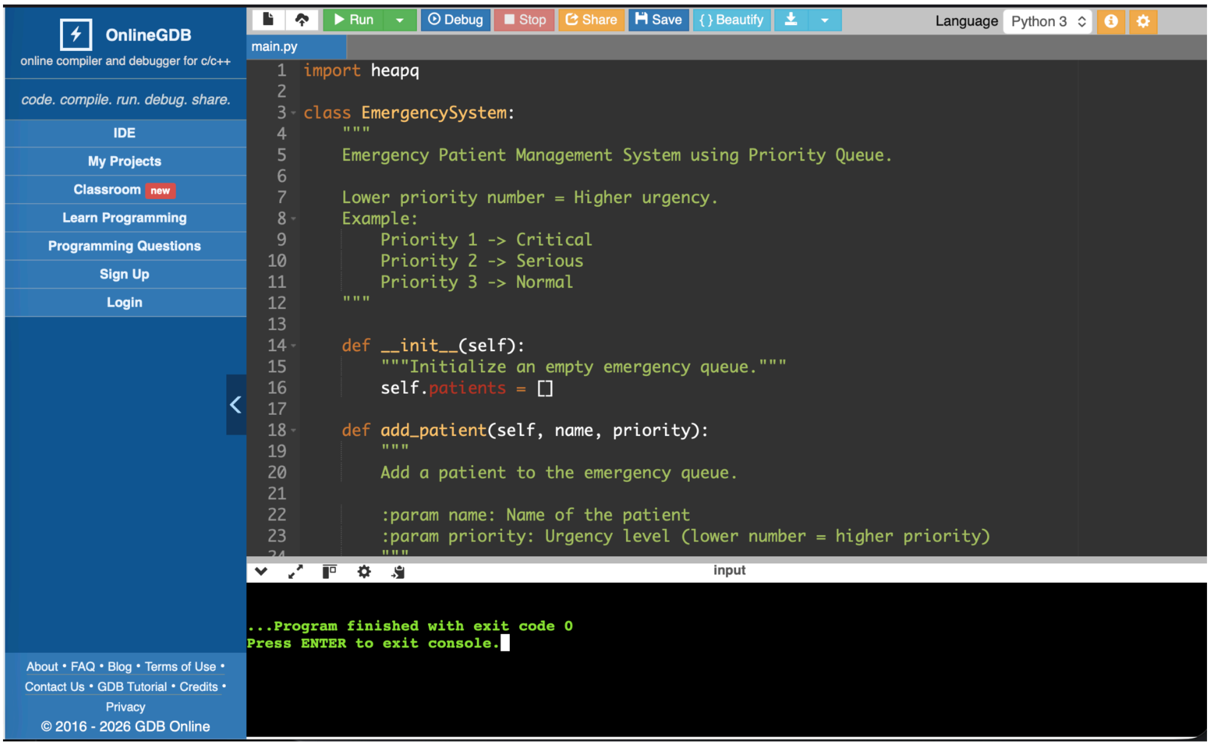
Expected Output:

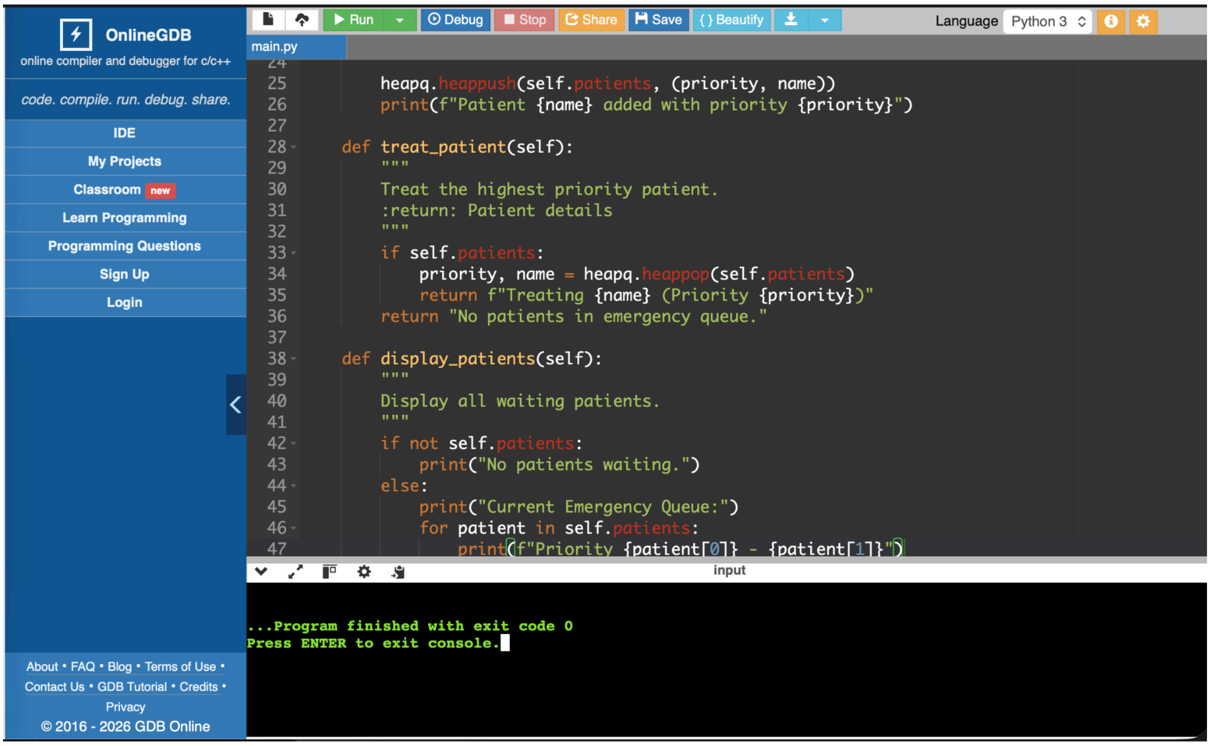
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code

```
import heapq

class EmergencySystem:
    """
    Emergency Patient Management System using Priority Queue.

    Lower priority number = Higher urgency.
    Example:
        Priority 1 -> Critical
        Priority 2 -> Serious
        Priority 3 -> Normal
    """

    def __init__(self):
        """Initialize an empty emergency queue."""
        self.patients = []

    def add_patient(self, name, priority):
        """
        Add a patient to the emergency queue.
```

```
        :param name: Name of the patient
        :param priority: Urgency level (lower number = higher priority)
        """
        heapq.heappush(self.patients, (priority, name))
        print(f"Patient {name} added with priority {priority}")

    def treat_patient(self):
        """
        Treat the highest priority patient.
        :return: Patient details
        """
        if self.patients:
            priority, name = heapq.heappop(self.patients)
            return f"Treating {name} (Priority {priority})"
        return "No patients in emergency queue."

    def display_patients(self):
        """
        Display all waiting patients.
        """
        if not self.patients:
            print("No patients waiting.")
        else:
            print("Current Emergency Queue:")
            for patient in self.patients:
                print(f"Priority {patient[0]} - {patient[1]}")
```





ask Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

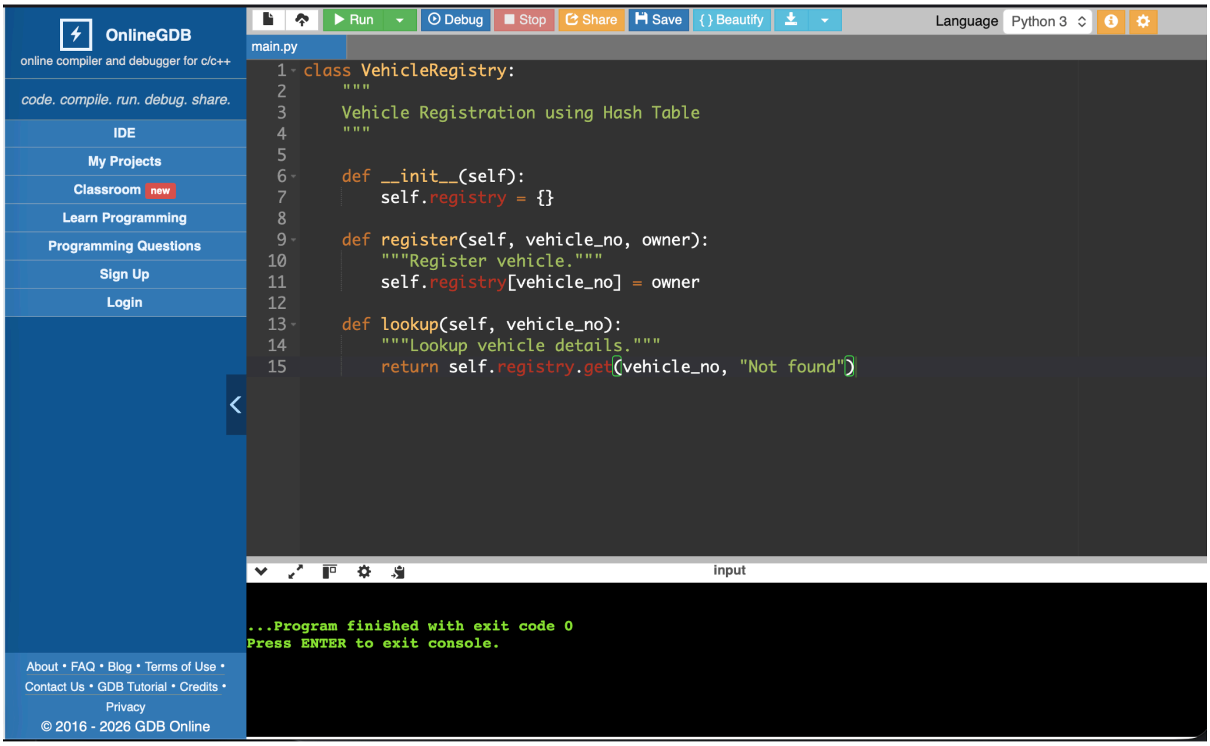
Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code

```
class VehicleRegistry:
    """
    Vehicle Registration using Hash Table
    """

    def __init__(self):
        self.registry = {}

    def register(self, vehicle_no, owner):
        """Register vehicle."""
        self.registry[vehicle_no] = owner

    def lookup(self, vehicle_no):
        """Lookup vehicle details."""
        return self.registry.get(vehicle_no, "Not found")
```




```
class OrderSystem:
    """
    Order Processing using Queue
    """

    def __init__(self):
        self.orders = []

    def place_order(self, order_id):
        """Add new order."""
        self.orders.append(order_id)

    def process_order(self):
        """Process first order."""
        if self.orders:
            return self.orders.pop(0)
        return "No orders"
```

Explanation

The queue follows the FIFO (First In First Out) principle.
enqueue() → adds elements at the end using append().
dequeue() → removes elements from the front using pop(0).
peek() → checks the first element without removing it.
size() → returns the total number of elements.
⏱ Time Complexity
enqueue() → O(1)
dequeue() → O(n) (because pop(0) shifts elements)

peek() → O(1)
size() → O(1)